

B-Spline Approximation via Kolmogorov–Arnold Networks

Motivation

Article

Advancing mathematics by guiding human intuition with AI

<https://doi.org/10.1038/s41586-021-04086-x>

Received: 10 July 2021

Accepted: 30 September 2021

Published online: 1 December 2021

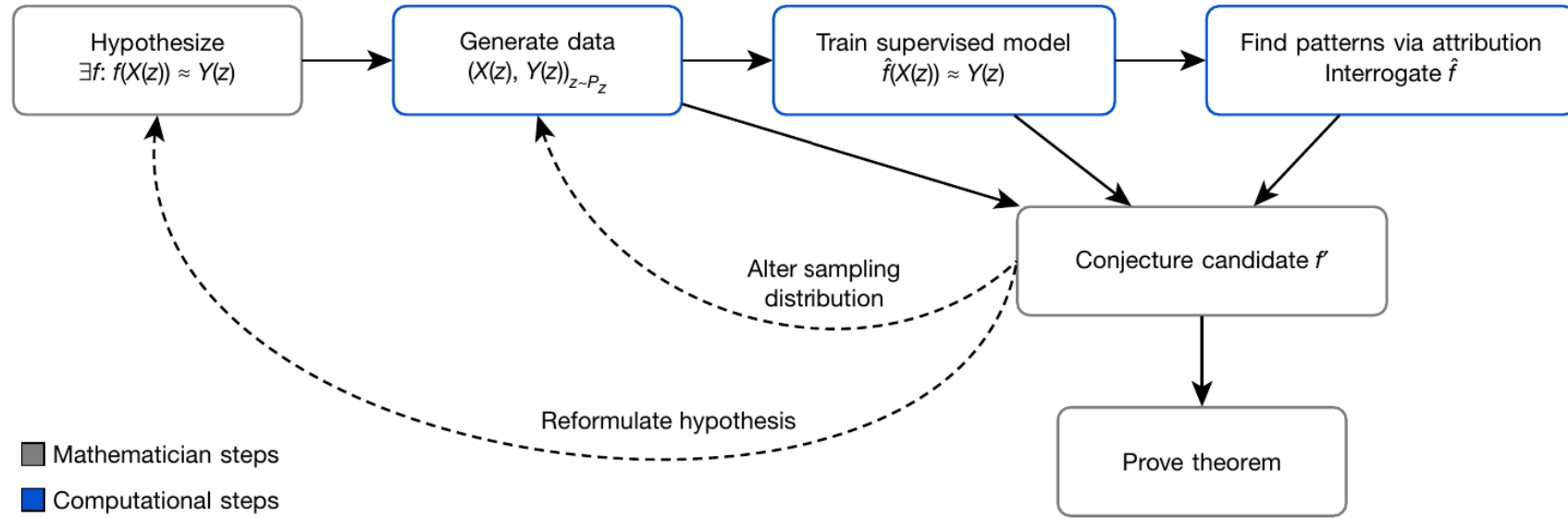
Open access



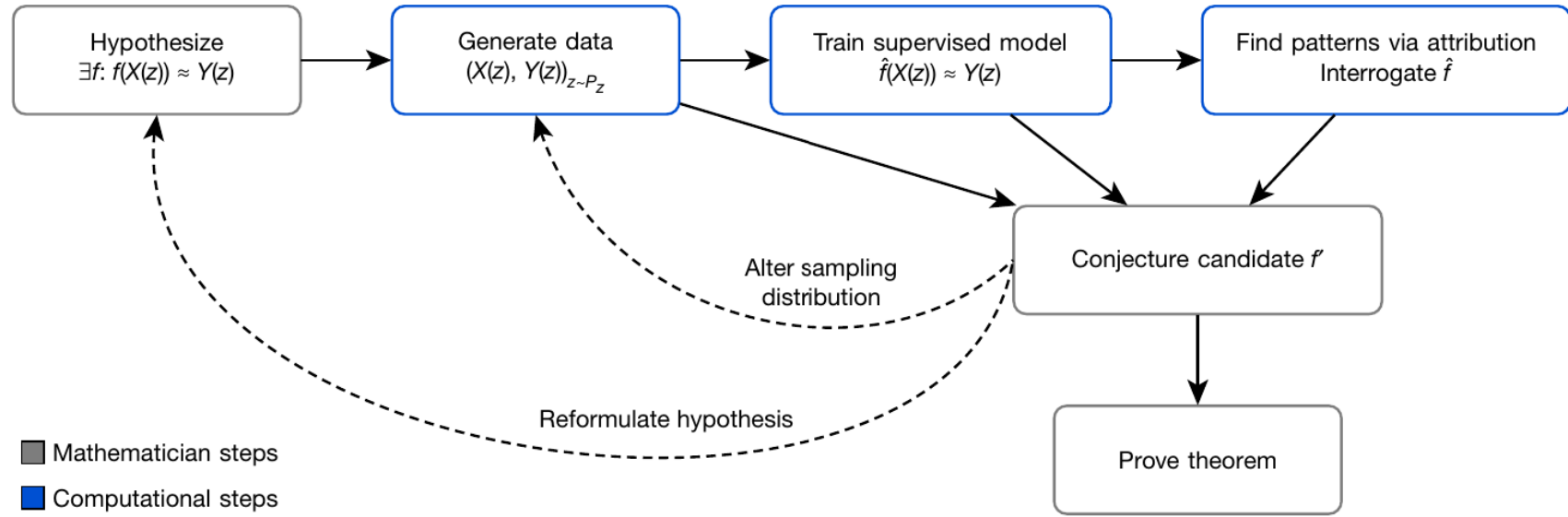
Check for updates

Alex Davies^{1✉}, Petar Veličković¹, Lars Buesing¹, Sam Blackwell¹, Daniel Zheng¹, Nenad Tomašev¹, Richard Tanburn¹, Peter Battaglia¹, Charles Blundell¹, András Juhász², Marc Lackenby², Georgie Williamson³, Demis Hassabis¹ & Pushmeet Kohli^{1✉}

The practice of **mathematics** involves **discovering patterns** and using these to **formulate and prove conjectures**, resulting in theorems. Since the 1960s, mathematicians have used **computers** to assist in the discovery of patterns and formulation of conjectures¹, most famously in the Birch and Swinnerton-Dyer conjecture², a Millennium Prize Problem³. Here we provide examples of **new fundamental results in pure mathematics** that have been discovered with the assistance of **machine learning**—demonstrating a method by which machine learning can aid mathematicians in discovering new conjectures and theorems. We propose a



z mathematical object
 $X(z)$ features/input
 $Y(z)$ target/output
 $f : X(z) \mapsto Y(z)$ relationship

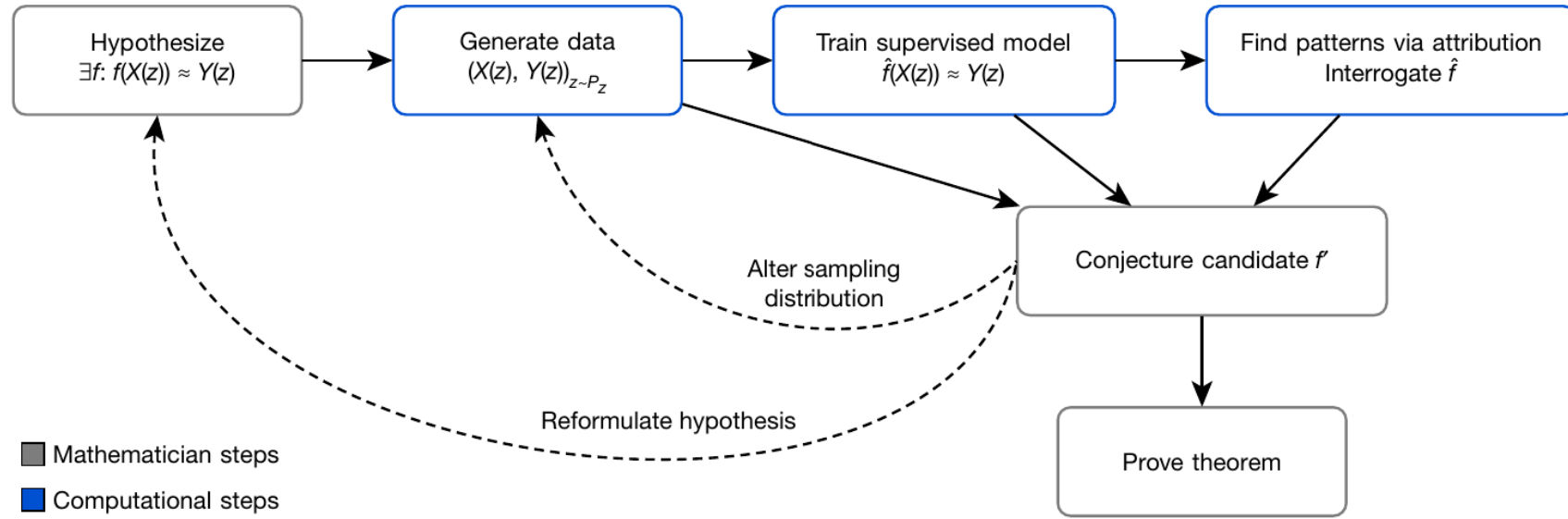


z = convex polyhedra

$X(z) = (\# \text{vertices}, \# \text{edges}, \text{volume}, \text{surface area}) \in \mathbb{Z} \times \mathbb{Z} \times \mathbb{R} \times \mathbb{R}$

$Y(z) = (\# \text{faces}) \in \mathbb{Z}$

Euler's formula: $V - E + F = 2 \rightsquigarrow X(z) \cdot (-1, 1, 0, 0) + 2 = Y(z)$



z mathematical object
 $X(z)$ features/input
 $Y(z)$ target/output
 $f : X(z) \mapsto Y(z)$ relationship

$\hat{f}$ model
 $\hat{f}(X(z))$ prediction
 training = iteratively construct more accurate $\hat{f}$
 supervised = $Y(z)$ available
 attribution = which features are important for prediction

z: Knot	X(z): Geometric invariants				Y(z): Algebraic invariants		
	Volume	Chern–Simons	Meridional translation	...	Signature	Jones polynomial	...
	2.0299	0	i	...	0	$t^{-2} - t^{-1} + 1 - t + t^2$	...
	2.8281	-0.1532	$0.7381 + 0.8831i$	...	-2	$t - t^2 + 2t^3 - t^4 + t^5 - t^6$	...
	3.1640	0.1560	$-0.7237 + 1.0160i$	...	0	$t^{-2} - t^{-1} + 2 - 2t + t^2 - t^3 + t^4$	...

Fig. 2 | Examples of invariants for three hyperbolic knots. We hypothesized that there was a previously undiscovered relationship between the geometric and algebraic invariants.

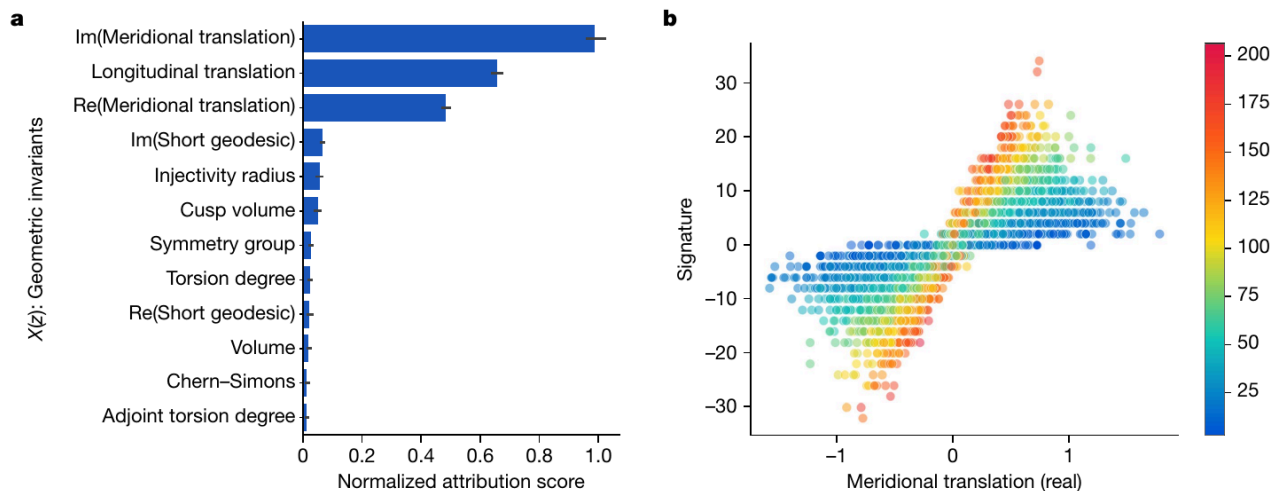


Fig. 3 | Knot theory attribution. **a**, Attribution values for each of the input $X(z)$. The features with high values are those that the learned function is most sensitive to and are probably relevant for further exploration. The 95%

confidence interval error bars are across 10 retrainings of the model.

b, Example visualization of relevant features—the real part of the meridional translation against signature, coloured by the longitudinal translation.

KAN: Kolmogorov–Arnold Networks

Ziming Liu^{1,4*} **Yixuan Wang**² **Sachin Vaidya**¹ **Fabian Ruehle**^{3,4}
James Halverson^{3,4} **Marin Soljačić**^{1,4} **Thomas Y. Hou**² **Max Tegmark**^{1,4}

¹ Massachusetts Institute of Technology

² California Institute of Technology

³ Northeastern University

⁴ The NSF Institute for Artificial Intelligence and Fundamental Interactions

different
architecture

smaller
interpretable
models

uses splines

interpretable = understand model / predictions / decisions

AI
+
Science

KAN 2.0: Kolmogorov-Arnold Networks Meet Science

NO
reverse
engineering

Ziming Liu^{1,4*} **Pingchuan Ma^{1,3}** **Yixuan Wang²** **Wojciech Matusik^{1,3}** **Max Tegmark^{1,4}**

¹ Massachusetts Institute of Technology

² California Institute of Technology

³ Computer Science and Artificial Intelligence Laboratory (CSAIL), MIT

⁴ The NSF Institute for Artificial Intelligence and Fundamental Interactions

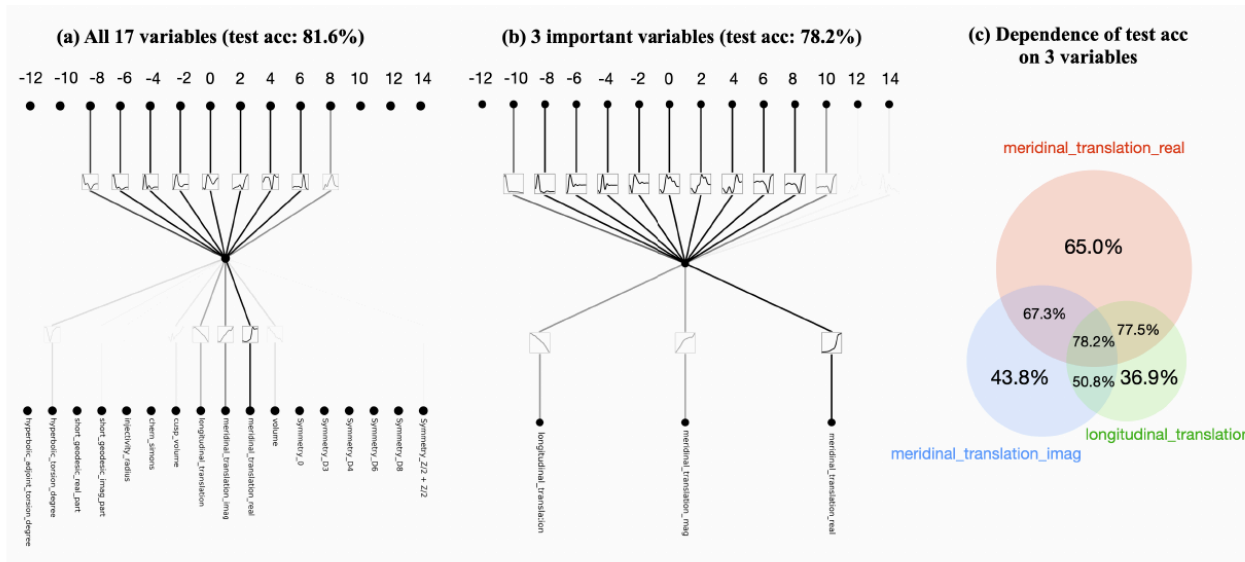


Figure 4.3: Knot dataset, supervised mode. With KANs, we rediscover Deepmind’s results that signature is mainly dependent on meridinal translation (real and imaginary parts).

Method	Architecture	Parameter Count	Accuracy
Deepmind’s MLP	4 layer, width-300	3×10^5	78.0%
KANs	2 layer, $[17, 1, 14]$ ($G = 3, k = 3$)	2×10^2	81.6%

Table 3: KANs can achieve better accuracy than MLPs with much fewer parameters in the signature classification problem. Soon after our preprint was first released, Prof. Shi Lab from Georgia tech discovered that an MLP with only 60 parameters is sufficient to achieve 80% accuracy (public but unpublished results). This is good news for AI + Science because this means perhaps many AI + Science tasks are not that computationally demanding than we might think (either with MLPs or with KANs), hence many new scientific discoveries are possible even on personal laptops.

Problem

B-Spline approximation

data points : $\{\mathbf{P}_i\}_{i=1}^N \subseteq \mathbb{R}^d$

$$\mathbf{S}(t) = \sum_{j=0}^n B_j^p(t) \cdot \mathbf{C}_j$$

- spline: $\mathbf{S}(t) \in \mathbb{R}^d$
- control points: $\{\mathbf{C}_j\}_{j=0}^n \subseteq \mathbb{R}^d$, $n := m - p - 1$
- parametrization: $\{t_i\}_{i=1}^N \subseteq \mathbb{R}$ such that $\mathbf{S}(t_i) \approx \mathbf{P}_i$
- knots: $\{u_h\}_{h=0}^m \subseteq \mathbb{R}$ non-decreasing

- basis functions (degree p): $B_j^p(t) \rightarrow$ Cox–de Boor recursion formula:

$$B_j^0(t) := \begin{cases} 1, & u_j \leq t \leq u_{j+1} \\ 0, & \text{otherwise} \end{cases}$$

$$B_j^p(t) := \frac{t - u_j}{u_{j+p} - u_j} B_j^{p-1}(t) + \frac{u_{j+p+1} - t}{u_{j+p+1} - u_{j+1}} B_{j+1}^{p-1}(t) , \quad j = 0, \dots, \underbrace{m - p - 1}_n$$

given knots $\mathbf{U} = (u_0, \dots, u_m)$ and parametrization $\mathbf{T} = (t_1, \dots, t_N)$,
compute optimal control points $\mathbf{C}$ by solving

$$\min \sum_{i=1}^N \|\mathbf{P}_i - \mathbf{S}(t_i)\|_2^2 = \min \|\mathbf{P} - \mathbf{BC}\|_F^2$$

given knots $\mathbf{U} = (u_0, \dots, u_m)$ and parametrization $\mathbf{T} = (t_1, \dots, t_N)$,
compute optimal control points $\mathbf{C}$ by solving

$$\min \sum_{i=1}^N \|\mathbf{P}_i - \mathbf{S}(t_i)\|_2^2 = \min \|\mathbf{P} - \mathbf{B}\mathbf{C}\|_F^2$$

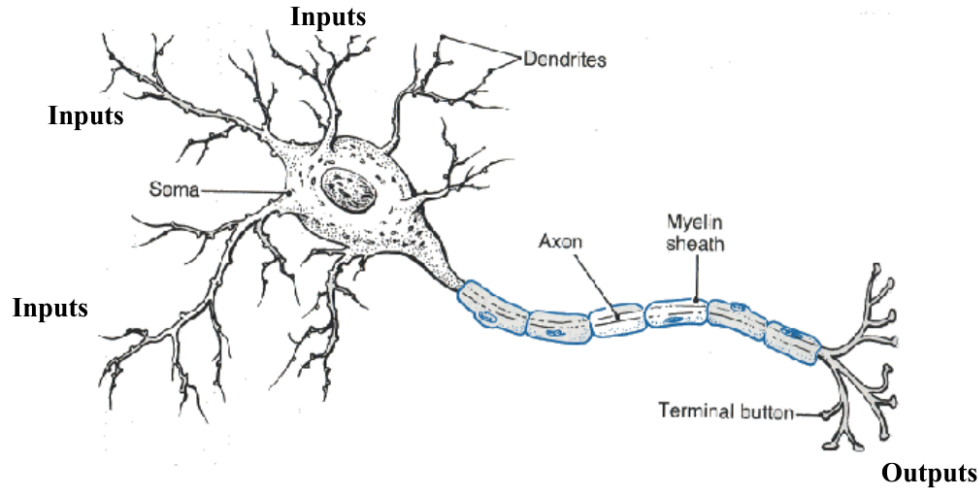
equivalent to the normal equation $(\mathbf{B}^T \mathbf{B})\mathbf{C} = \mathbf{B}^T \mathbf{P}$
where $\mathbf{B}$ depends on $\mathbf{U}$ and $\mathbf{T}$

$$\mathbf{B} := \begin{pmatrix} B_0^p(t_1) & \dots & B_n^p(t_1) \\ \vdots & \ddots & \vdots \\ B_0^p(t_N) & \dots & B_n^p(t_N) \end{pmatrix} \in \mathbb{R}^{N \times (n+1)}, \quad \mathbf{C} = \begin{pmatrix} \mathbf{C}_0 \\ \vdots \\ \mathbf{C}_n \end{pmatrix} \in \mathbb{R}^{(n+1) \times d}, \quad \mathbf{P} = \begin{pmatrix} \mathbf{P}_1 \\ \vdots \\ \mathbf{P}_N \end{pmatrix} \in \mathbb{R}^{N \times d}$$

(Vanilla) Neural Network

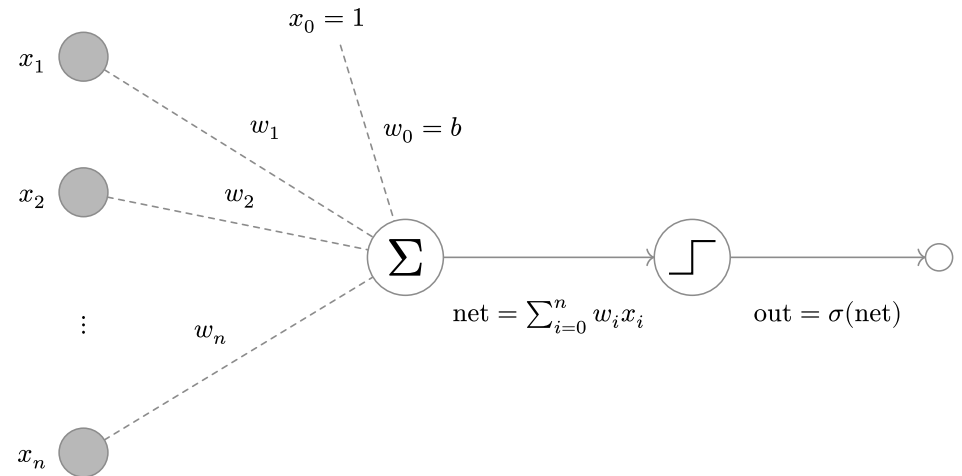
a.k.a. fully-connected feedforward network or multi-layer perceptron

Neurons



Biological neuron:

- dendrites: receive signals from other neurons
- soma: integrate incoming signals
- if signal exceeds threshold $\rightarrow$ spike
- axon: transmit output signal to other neurons



Artificial neuron (perceptron):

- inputs: $x_1, \dots, x_n$ (features)
- weights: $w_1, \dots, w_n$ (learned parameters)
- bias: b (learned parameter)
- activation function: σ (non-linear function)
- output: $y = \sigma(w_1 x_1 + \dots + w_n x_n + b)$

Single-Layer Perceptron

Perceptron / Linear Threshold Unit (LTU): $f(\boldsymbol{x}) = h(\boldsymbol{w} \cdot \boldsymbol{x} + b)$

where $h(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$ is the Heaviside step function

Single-Layer Perceptron

Perceptron / Linear Threshold Unit (LTU): $f(\mathbf{x}) = h(\mathbf{w} \cdot \mathbf{x} + b)$

where $h(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$ is the Heaviside step function

it's equivalent to a linear classifier:

$$f(\mathbf{x}) = 1 \text{ if } \mathbf{w} \cdot \mathbf{x} + b > 0 \text{ else } f(\mathbf{x}) = 0$$

decision boundary: $\mathbf{w} \cdot \mathbf{x} + b = 0$ (a hyperplane in $\mathbb{R}^n$)

Single-Layer Perceptron

Perceptron / Linear Threshold Unit (LTU): $f(\mathbf{x}) = h(\mathbf{w} \cdot \mathbf{x} + b)$

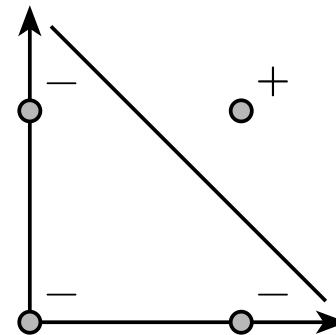
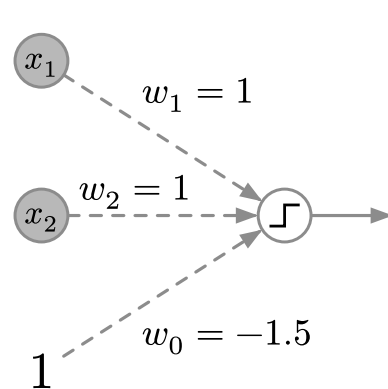
where $h(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$ is the Heaviside step function

it's equivalent to a linear classifier:

$$f(\mathbf{x}) = 1 \text{ if } \mathbf{w} \cdot \mathbf{x} + b > 0 \text{ else } f(\mathbf{x}) = 0$$

decision boundary: $\mathbf{w} \cdot \mathbf{x} + b = 0$ (a hyperplane in $\mathbb{R}^n$)

x_1	x_2	$f(\mathbf{x})$
0	0	0
0	1	0
1	0	0
1	1	1



AND

Single-Layer Perceptron

Perceptron / Linear Threshold Unit (LTU): $f(\mathbf{x}) = h(\mathbf{w} \cdot \mathbf{x} + b)$

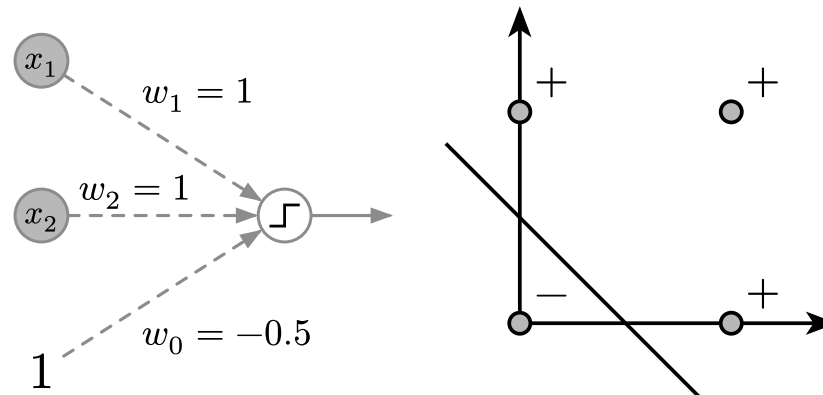
where $h(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$ is the Heaviside step function

it's equivalent to a linear classifier:

$$f(\mathbf{x}) = 1 \text{ if } \mathbf{w} \cdot \mathbf{x} + b > 0 \text{ else } f(\mathbf{x}) = 0$$

decision boundary: $\mathbf{w} \cdot \mathbf{x} + b = 0$ (a hyperplane in $\mathbb{R}^n$)

x_1	x_2	$f(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	1



OR

Single-Layer Perceptron

Perceptron / Linear Threshold Unit (LTU): $f(\mathbf{x}) = h(\mathbf{w} \cdot \mathbf{x} + b)$

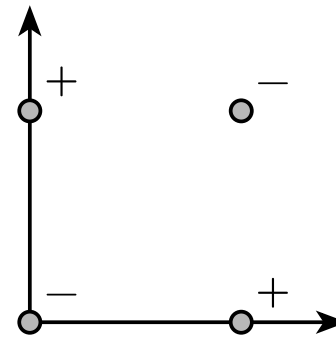
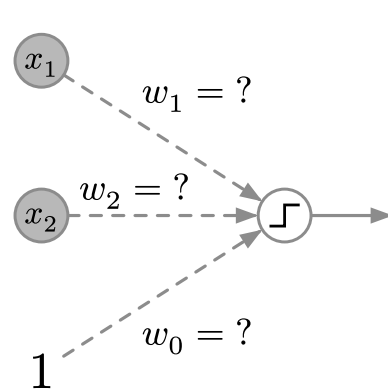
where $h(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$ is the Heaviside step function

it's equivalent to a linear classifier:

$$f(\mathbf{x}) = 1 \text{ if } \mathbf{w} \cdot \mathbf{x} + b > 0 \text{ else } f(\mathbf{x}) = 0$$

decision boundary: $\mathbf{w} \cdot \mathbf{x} + b = 0$ (a hyperplane in $\mathbb{R}^n$)

x_1	x_2	$f(\mathbf{x})$
0	0	0
0	1	1
1	0	1
1	1	0

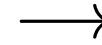


XOR

NOT linearly separable!

XOR problem

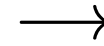
A single-layer perceptron cannot learn the XOR function because the XOR function is not linearly separable.



Minsky & Papert,
Perceptrons (1969)

XOR problem

A single-layer perceptron cannot learn the XOR function because the XOR function is not linearly separable.



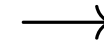
Minsky & Papert,
Perceptrons (1969)

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2 = \underbrace{\overline{(x_1 \cdot x_2)}}_{h_1} \cdot \underbrace{(x_1 + x_2)}_{h_2}$$

$\cdot = \text{AND}$ $\overline{x} = \text{NOT}$
 $+= \text{OR}$ $\oplus = \text{XOR}$

XOR problem

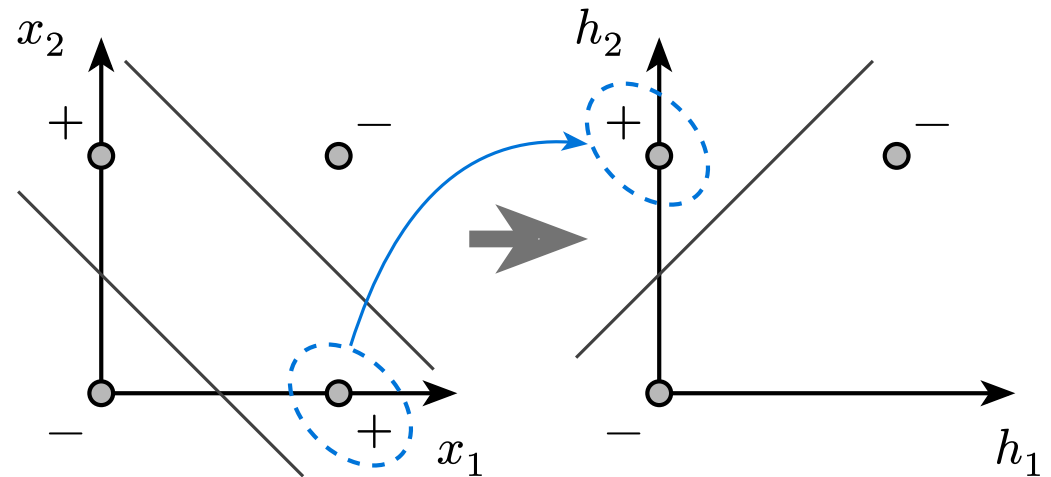
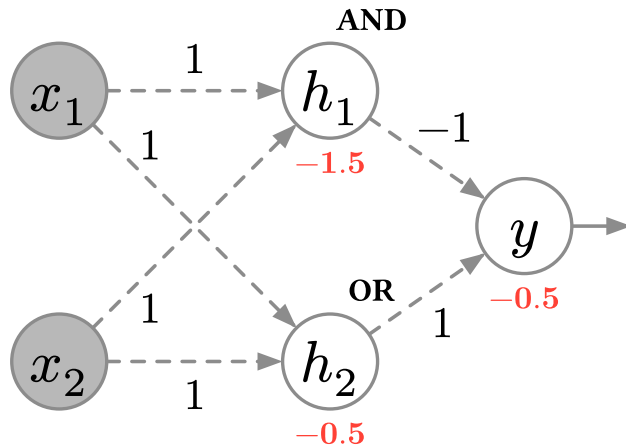
A single-layer perceptron cannot learn the XOR function because the XOR function is not linearly separable.



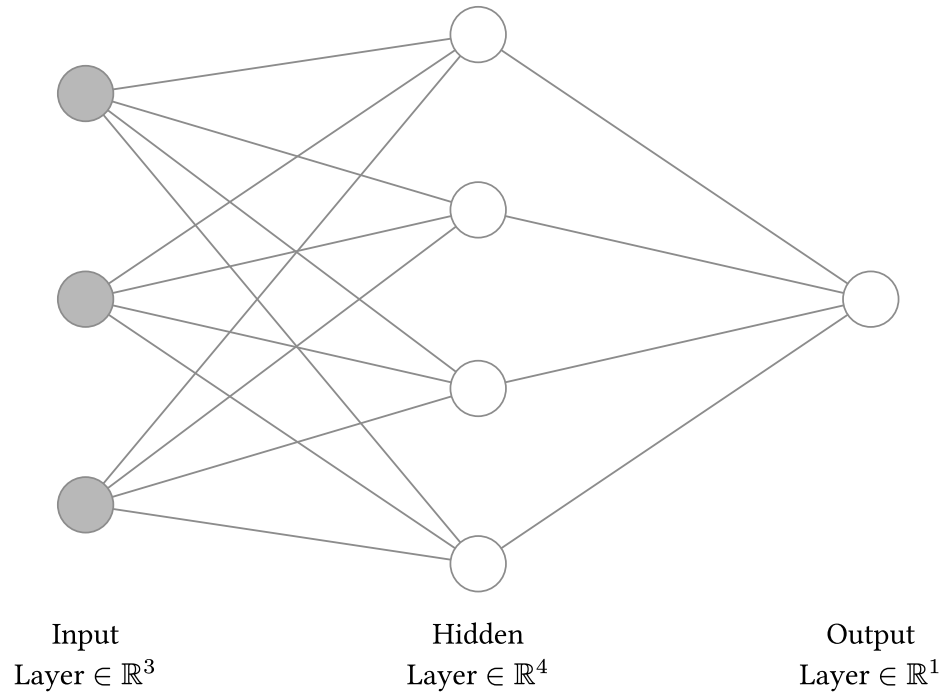
Minsky & Papert,
Perceptrons (1969)

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2 = \underbrace{(x_1 \cdot x_2)}_{h_1} \cdot \underbrace{(\overline{x_1} + \overline{x_2})}_{h_2}$$

$\cdot$ = AND $\overline{x}$ = NOT
 $+$ = OR $\oplus$ = XOR



Shallow Multi-Layer Perceptron

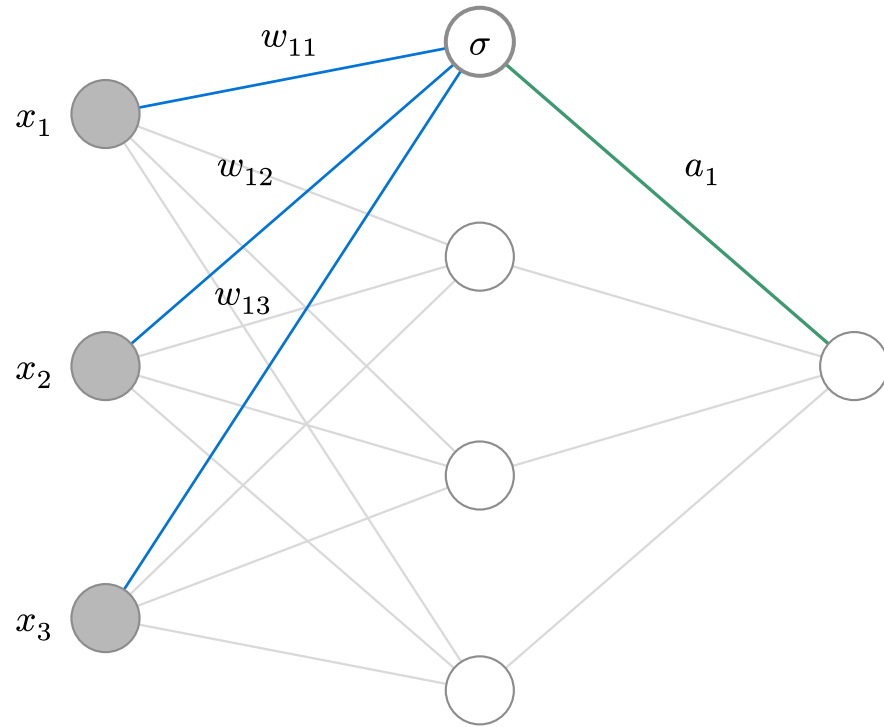


$$f(\mathbf{x}) = \sum_{i=1}^{N_{\text{hidden}}} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$$

$$\mathbf{x} = (x_1, \dots, x_n)$$

$$\mathbf{w}_i = (w_{i1}, \dots, w_{in})$$

Shallow Multi-Layer Perceptron

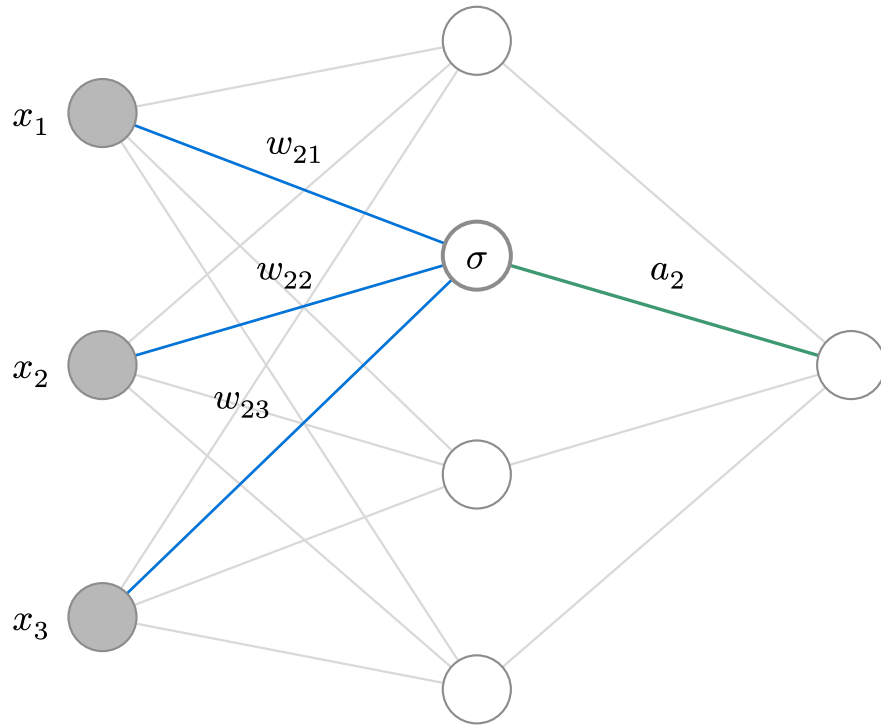


$$f(\mathbf{x}) = \sum_{i=1}^{N_{\text{hidden}}} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$$

$$\mathbf{x} = (x_1, \dots, x_n)$$

$$\mathbf{w}_i = (w_{i1}, \dots, w_{in})$$

Shallow Multi-Layer Perceptron

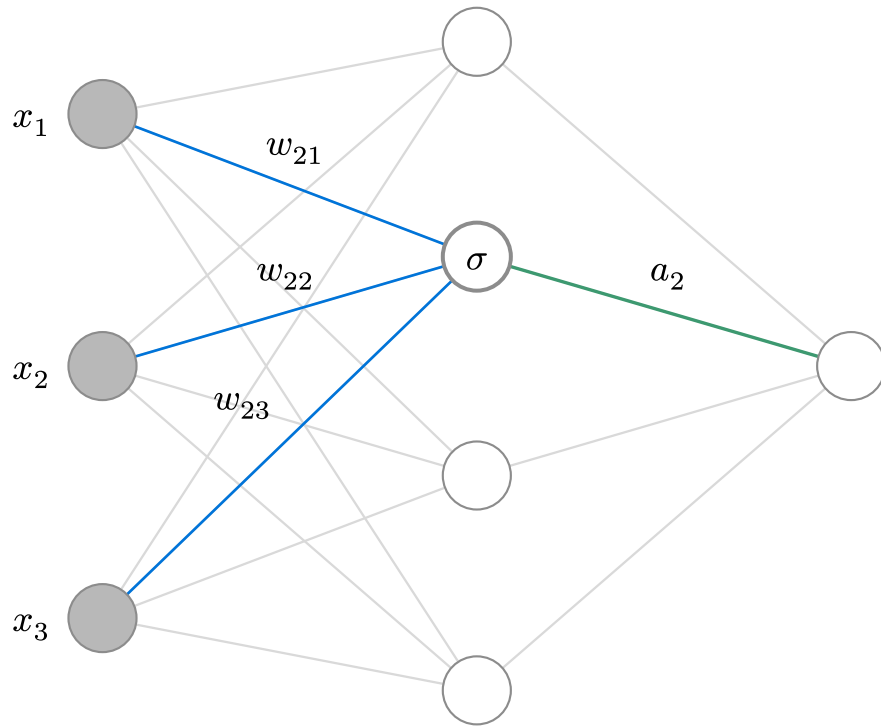


$$f(\mathbf{x}) = \sum_{i=1}^{N_{\text{hidden}}} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$$

$$\mathbf{x} = (x_1, \dots, x_n)$$

$$\mathbf{w}_i = (w_{i1}, \dots, w_{in})$$

Shallow Multi-Layer Perceptron



$$f(\mathbf{x}) = \sum_{i=1}^{N_{\text{hidden}}} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$$

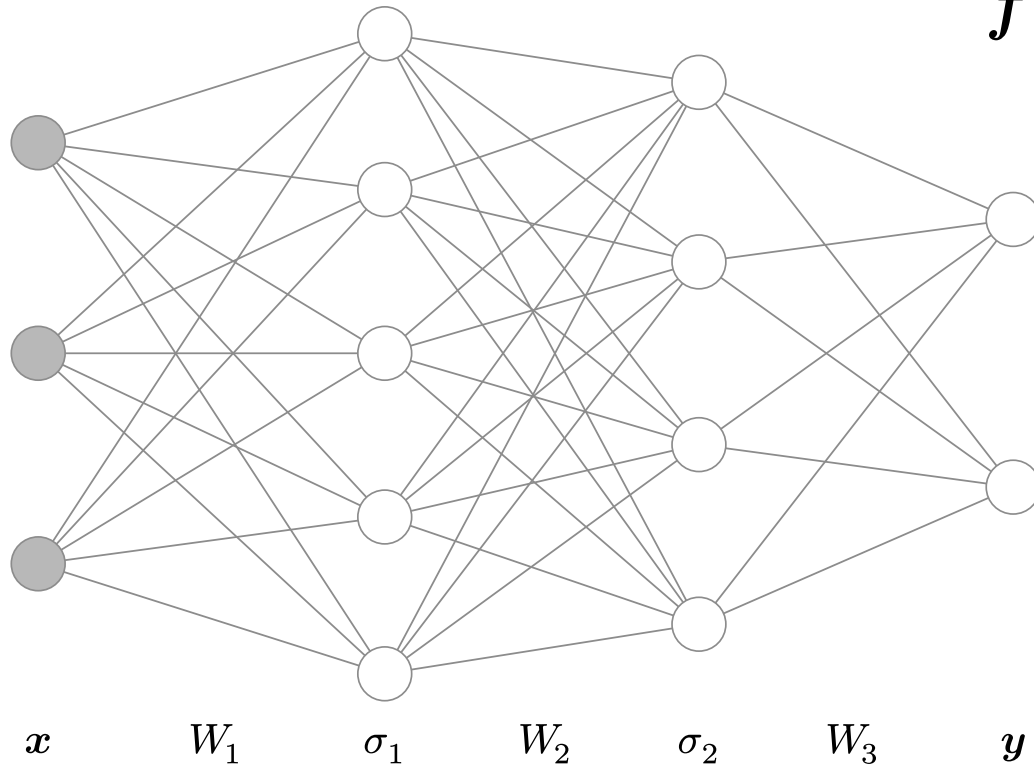
$$\mathbf{x} = (x_1, \dots, x_n)$$

$$\mathbf{w}_i = (w_{i1}, \dots, w_{in})$$

“Universal approximation theorem”

a single hidden layer with sufficiently many neurons can approximate any continuous function on compact subsets of $\mathbb{R}^n$ to any desired accuracy, given appropriate activation functions (e.g., sigmoid, ReLU)

Deep Multi-Layer Perceptron



$$f(x) = (W_L \circ \sigma_{L-1} \circ \dots \circ \sigma_1 \circ W_1)(x)$$

$$x = (x_1, \dots, x_n)$$

$$W_l = \begin{pmatrix} w_{11}^{(l)} & \dots & w_{1N_{l-1}}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{N_l 1}^{(l)} & \dots & w_{N_l N_{l-1}}^{(l)} \end{pmatrix}$$

N_l : # neurons in layer l

(Supervised) Learning / Training

how to find weights? (don't know the true f , want to “learn” it from data)

(Supervised) Learning / Training

how to find weights? (don't know the true f , want to “learn” it from data)

f_{θ} : statistical model, parametric function [network]

θ : parameters [weights and biases]

$\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$: (supervised) dataset, (input, label) pairs

$\hat{\mathbf{y}} = f_{\theta}(\mathbf{x})$: prediction

$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$: loss function, measures error between true label and prediction

(Supervised) Learning / Training

how to find weights? (don't know the true f , want to “learn” it from data)

f_{θ} : statistical model, parametric function [network]

θ : parameters [weights and biases]

$\{(x_i, y_i)\}_{i=1}^N$: (supervised) dataset, (input, label) pairs

$\hat{y} = f_{\theta}(x)$: prediction

$\mathcal{L}(y, \hat{y})$: loss function, measures error between true label and prediction

Idea: adjust parameters θ to minimize error on training data

$$\min_{\theta \in \Theta} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(x_i))$$

(Supervised) Learning / Training

training $\longleftrightarrow$ optimization over parameter space

(Supervised) Learning / Training

training $\longleftrightarrow$ optimization over parameter space



- very complicated
- non-linear/convex
- high-dimensional

(Supervised) Learning / Training

training $\longleftrightarrow$ optimization over parameter space

very complicated
non-linear/convex
high-dimensional

if f_{θ} is differentiable w.r.t. θ , we can use gradient descent

$$\theta^{k+1} := \theta^k - \eta \nabla_{\theta} \mathcal{E}(\theta^k) , \quad \mathcal{E}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(x_i))$$

η : learning rate, step size for parameter updates

variants: SGD / mini-batch, momentum, adaptive lr etc.

(Supervised) Learning / Training

training $\longleftrightarrow$ optimization over parameter space

very complicated
non-linear/convex
high-dimensional

if f_{θ} is differentiable w.r.t. θ , we can use gradient descent

$$\theta^{k+1} := \theta^k - \eta \nabla_{\theta} \mathcal{E}(\theta^k) , \quad \mathcal{E}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(x_i))$$

η : learning rate, step size for parameter updates

variants: SGD / mini-batch, momentum, adaptive lr etc.

how to compute gradients?

backpropagation: chain rule, but efficient (automatic differentiation)

Kolmogorov–Arnold Network (KAN)

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{(x+y)^2}{2}$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{z^2}{2} \qquad z = x + y$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{z^2}{2} \qquad z = x + y$$

$$f(x_1, x_2) = \qquad x_1 \quad + \qquad x_2 \quad + \qquad x_1 \quad + \qquad x_2 \quad + \dots + \qquad x_1 \quad + \qquad x_2$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{z^2}{2} \qquad z = x + y$$

$$f(x_1, x_2) = \phi_{1,1}(x_1) + \phi_{1,2}(x_2) + \phi_{2,1}(x_1) + \phi_{2,2}(x_2) + \dots + \phi_{k,1}(x_1) + \phi_{k,2}(x_2)$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{z^2}{2} \qquad z = x + y$$

$$f(x_1, x_2) = \Phi_1(\phi_{1,1}(x_1) + \phi_{1,2}(x_2)) + \Phi_2(\phi_{2,1}(x_1) + \phi_{2,2}(x_2)) + \dots + \Phi_k(\phi_{k,1}(x_1) + \phi_{k,2}(x_2))$$

Kolmogorov–Arnold representation theorem

Idea: multivariate is complicated (except +), but univariate is easy!
separate variables as much as possible ($\rightarrow$ easier derivatives)

$$f(x, y) = \phi_1(x) + \phi_2(y) \quad \boxtimes \boxtimes$$

$$xy = -\frac{x^2}{2} - \frac{y^2}{2} + \frac{z^2}{2} \qquad z = x + y$$

$$f(x_1, x_2) = \Phi_1(\underbrace{\phi_{1,1}(x_1) + \phi_{1,2}(x_2)}_{t_1}) + \Phi_2(\underbrace{\phi_{2,1}(x_1) + \phi_{2,2}(x_2)}_{t_2}) + \dots + \Phi_k(\underbrace{\phi_{k,1}(x_1) + \phi_{k,2}(x_2)}_{t_k})$$

Kolmogorov–Arnold representation theorem

Theorem.

Any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ can be represented as

$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

where $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ and $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ are continuous functions.

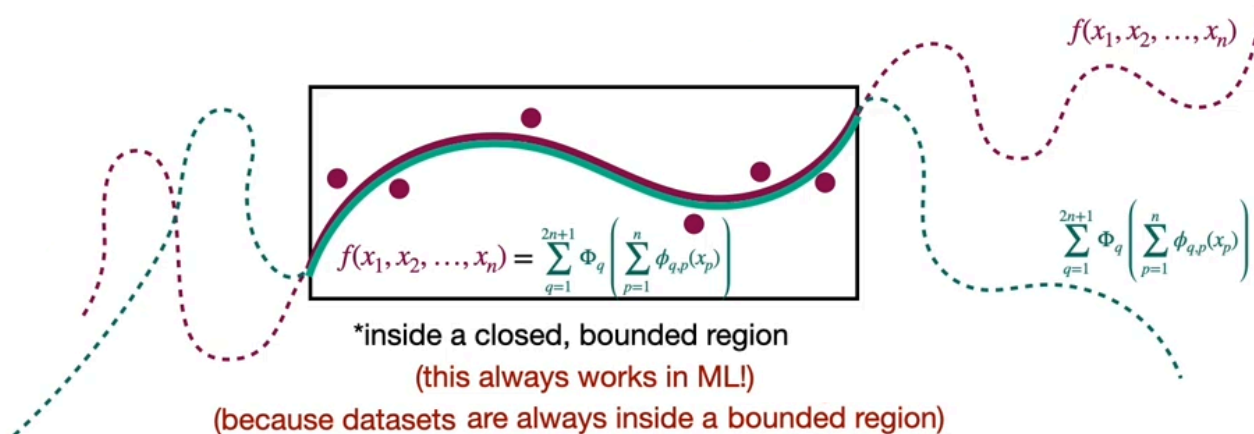
Kolmogorov–Arnold representation theorem

Theorem.

Any continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ can be represented as

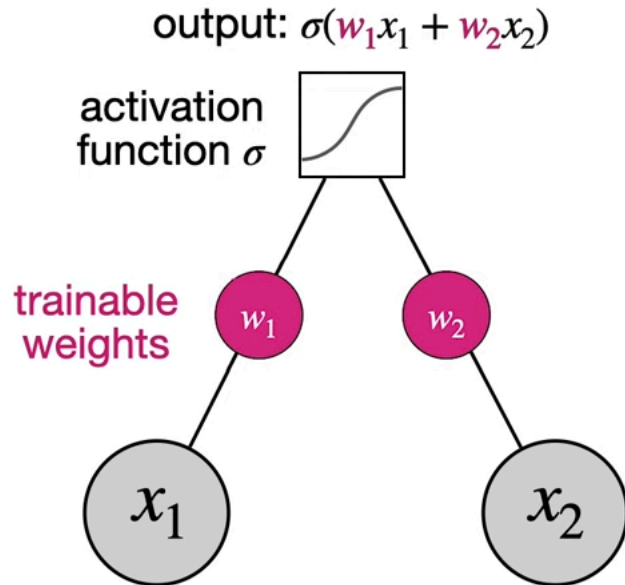
$$f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

where $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ and $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ are continuous functions.

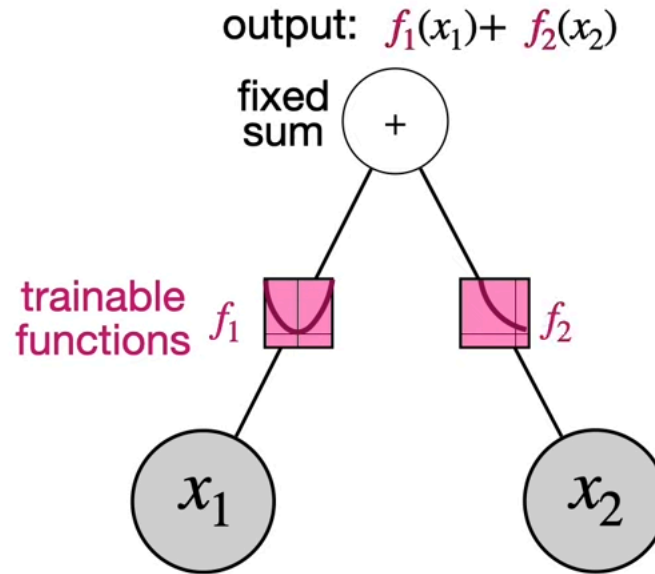


Kolmogorov–Arnold Network

MLP (Multi-Layer Perceptron)

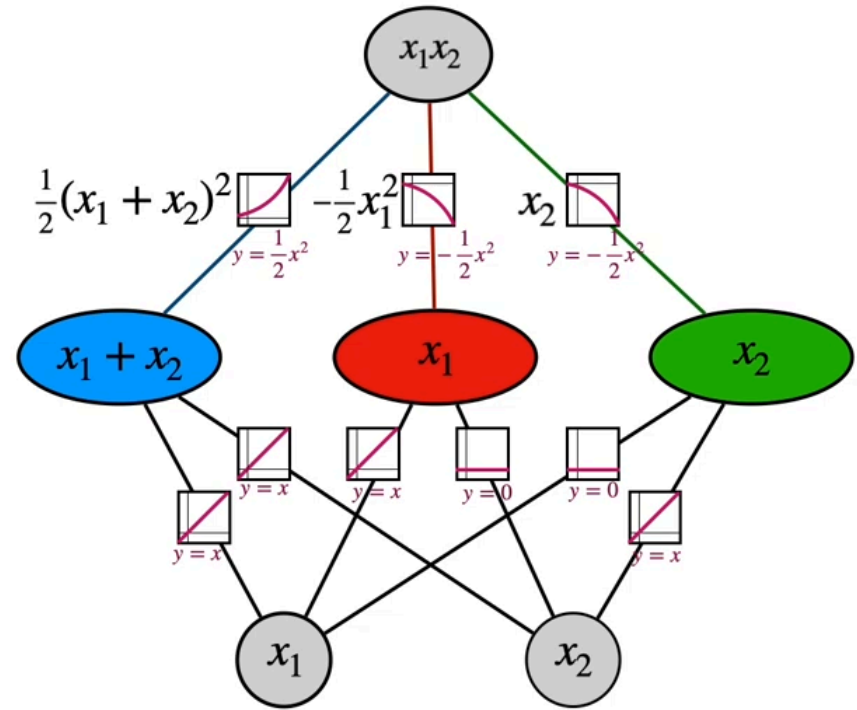


KAN (Kolmogorov-Arnold Network)



Kolmogorov–Arnold Network

2 layers are enough for
universal approximation!



$$x_1 x_2 = \frac{1}{2}(x_1 + x_2)^2 - \frac{1}{2}x_1^2 - \frac{1}{2}x_2^2$$

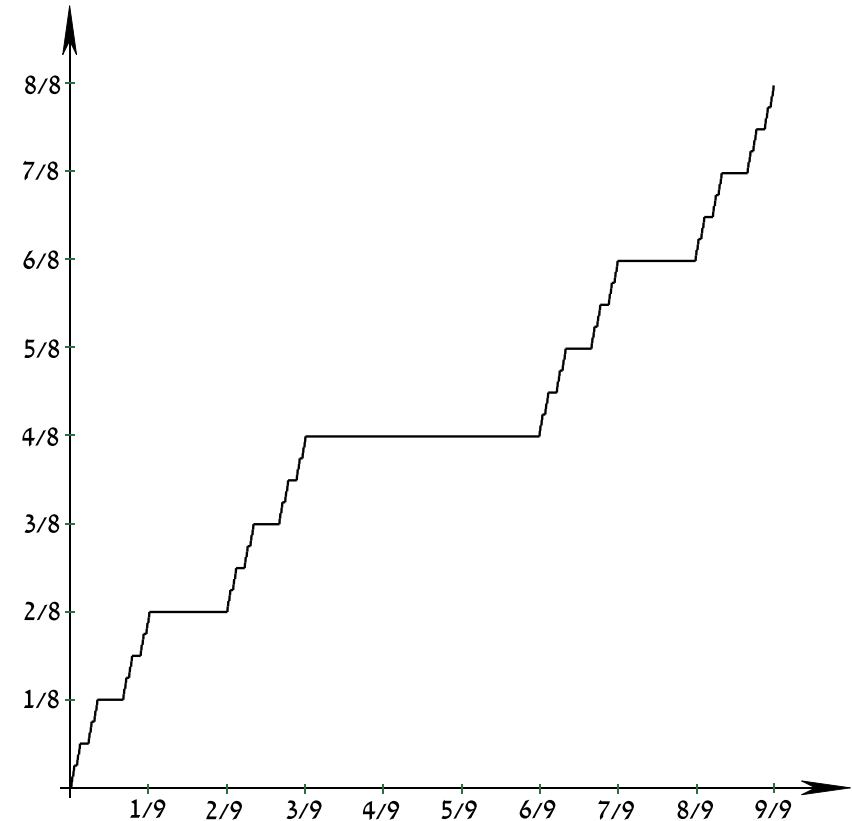
Kolmogorov–Arnold Network

2 layers are enough for
universal approximation!

but ϕ may be very “ugly”
(e.g., non-smooth, fractal-like)



not practical for ML
(Girosi & Poggio, 1989)



<https://commons.wikimedia.org/wiki/File:CantorEscalier-2.svg>

Kolmogorov–Arnold Network

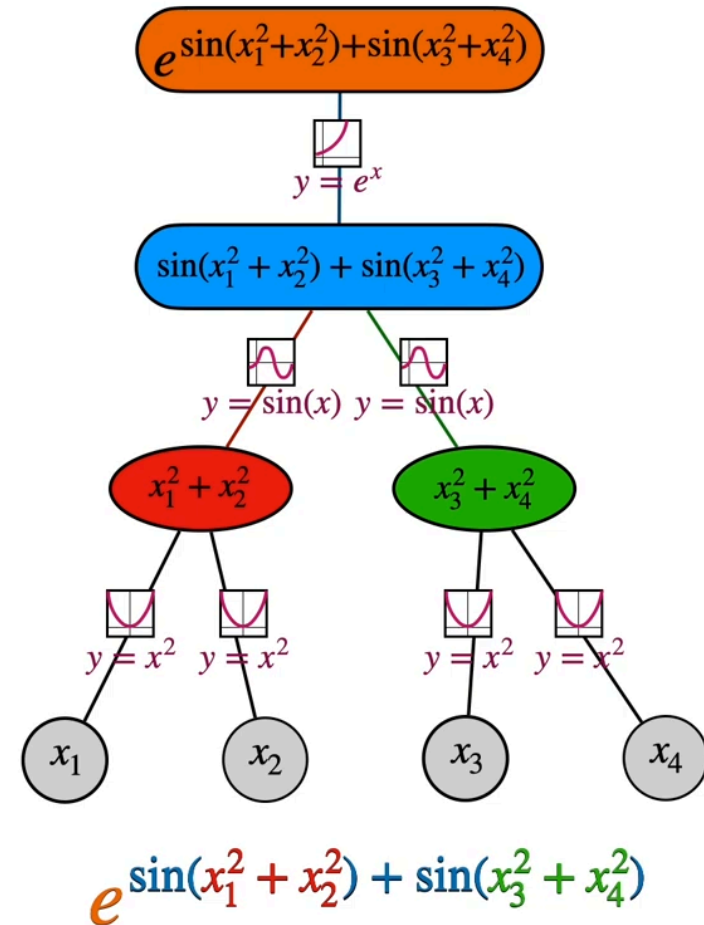
2 layers are enough for
universal approximation!

but ϕ may be very “ugly”
(e.g., non-smooth, fractal-like)



not practical for ML
(Girosi & Poggio, 1989)

Idea: go deeper!



Kolmogorov–Arnold Network

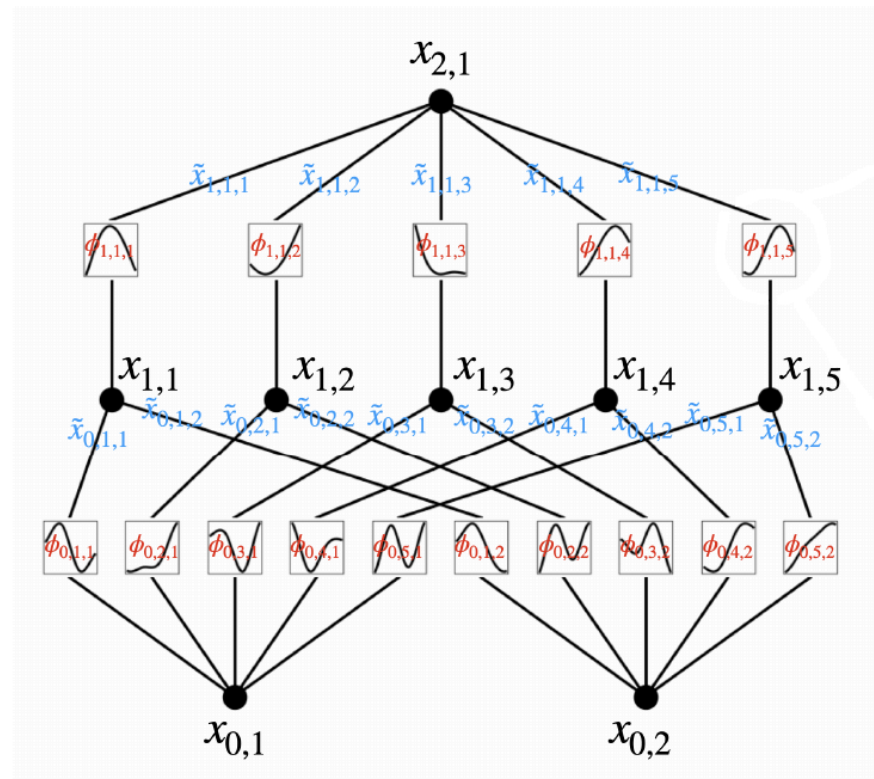
theorem $\rightarrow [n, 2n + 1, 1]$ KAN

$$f(\mathbf{x}) = (\Phi_L \circ \dots \circ \Phi_1)(\mathbf{x})$$

$$\Phi_l = \begin{pmatrix} \phi_{1,1}^{(l)}(\cdot) & \dots & \phi_{1,N_{l-1}}^{(l)}(\cdot) \\ \vdots & \ddots & \vdots \\ \phi_{N_l,1}^{(l)}(\cdot) & \dots & \phi_{N_l,N_{l-1}}^{(l)}(\cdot) \end{pmatrix}$$

arbitrary width KAN layer

$$x_j^{(l+1)} = \sum_{i=1}^{N_l} \phi_{j,i}^{(l+1)}(x_i^{(l)}), \quad j = 1, \dots, N_{l+1}$$

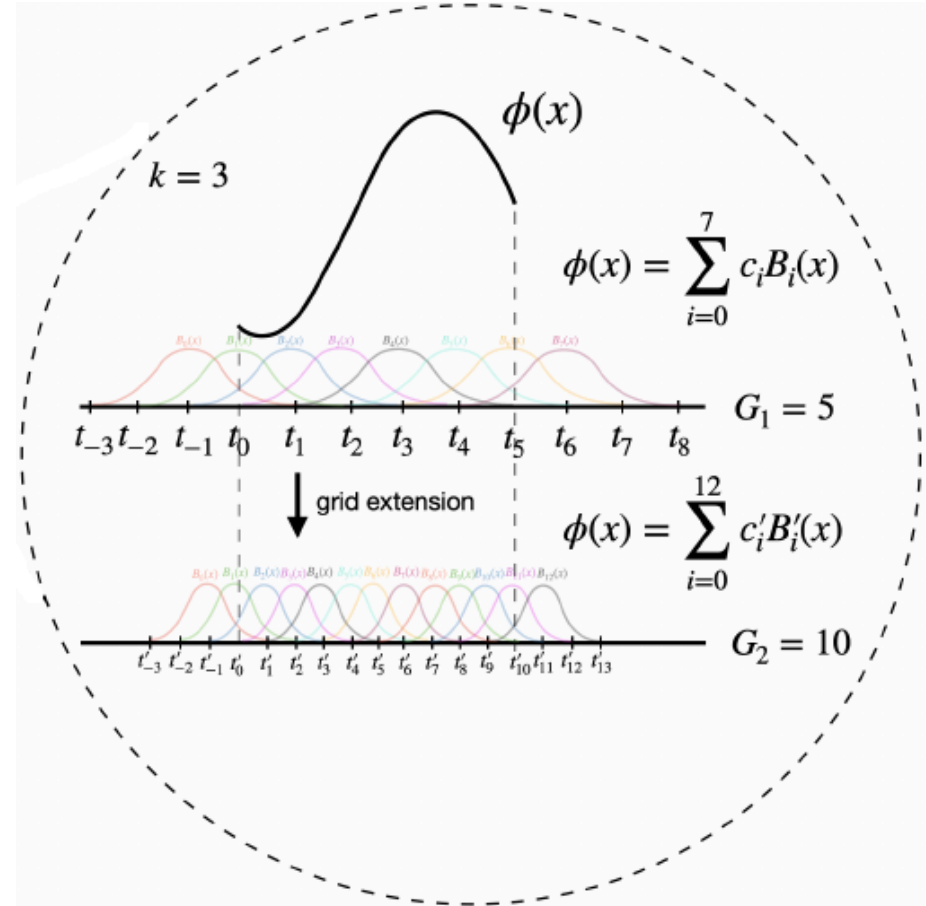


Kolmogorov–Arnold Network

how to parametrize ϕ 's ?

use 1D B-splines:

- flexible, can approximate any continuous function
- efficient to compute and differentiate
- need few parameters ($\#ctrls = G + k$)
- can be “fine-grained”



Interpretability

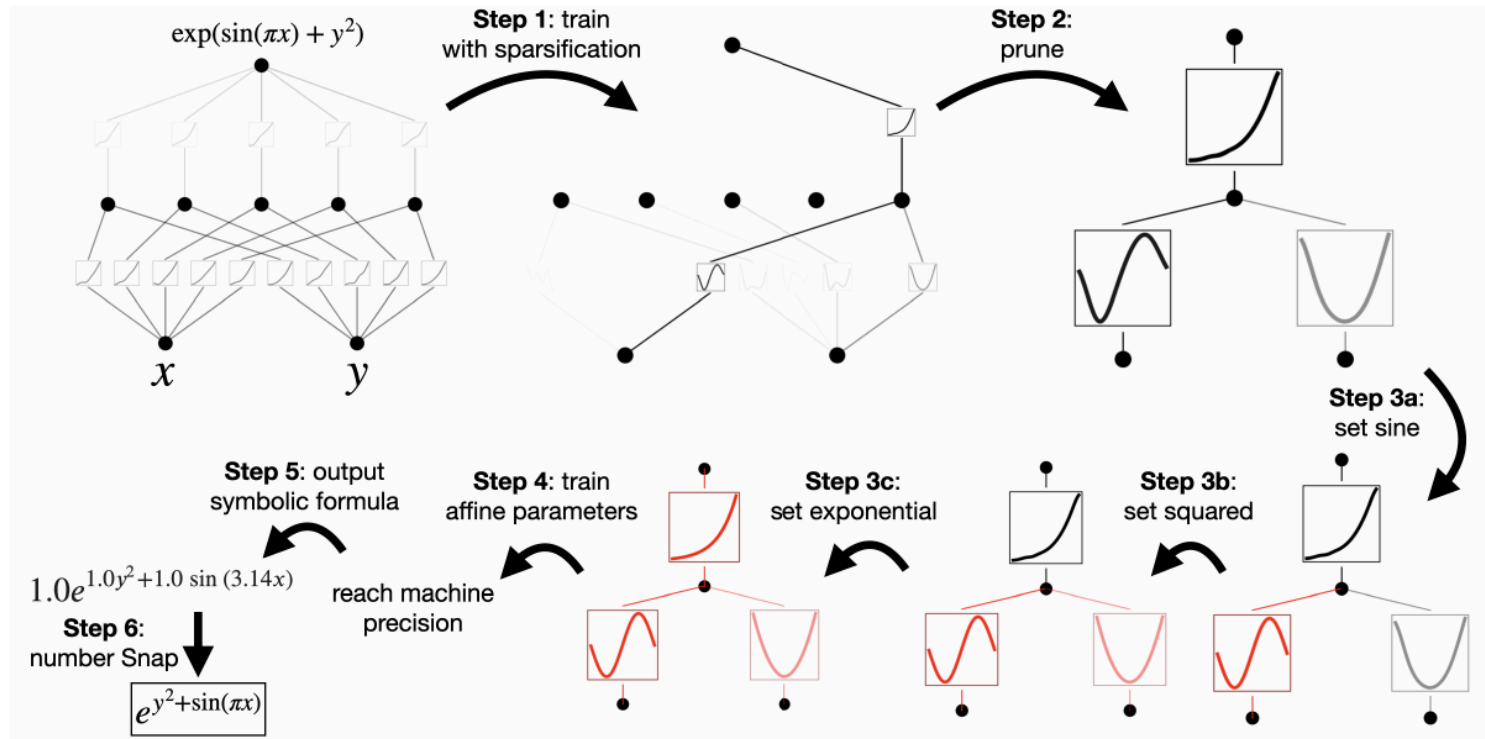


Figure 2.4: An example of how to do symbolic regression with KAN.

sparse: penalize
non-zero prms

visualize:
transparency

prune: smaller net

symbolic setting:

i/o shift/scaling

→ fitting aff. prms

$$y \approx cf(ax + b) + d$$

Experiments

Task

↑ problem

Goal: find optimal knot vector of a B-spline fitting data points

Task

↑ problem

Goal: find optimal knot vector of a B-spline fitting data points

For simplicity

- fixed degree $p = 3$ (cubic B-splines)
- fixed number of points, ordered
- fixed number of knots
- fixed parametrization (uniform)

Task

↑ problem

Goal: find optimal knot vector of a B-spline fitting data points

For simplicity

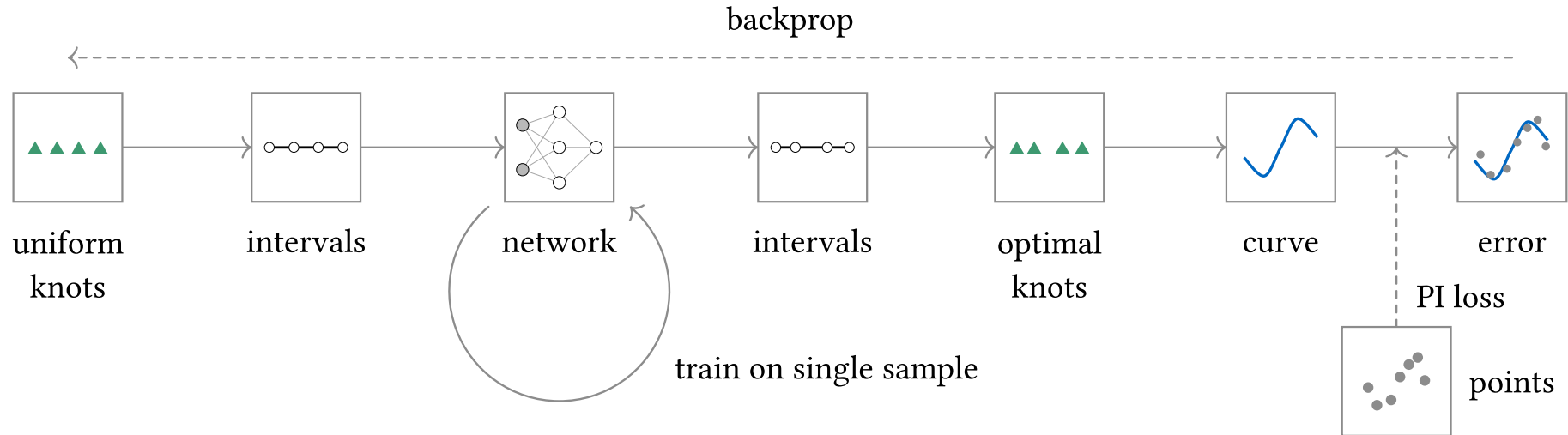
- fixed degree $p = 3$ (cubic B-splines)
- fixed number of points, ordered
- fixed number of knots
- fixed parametrization (uniform)

What we want?

- find relationships with features, e.g. curvature
- accuracy is not the priority (but still important)
- learn something about the problem

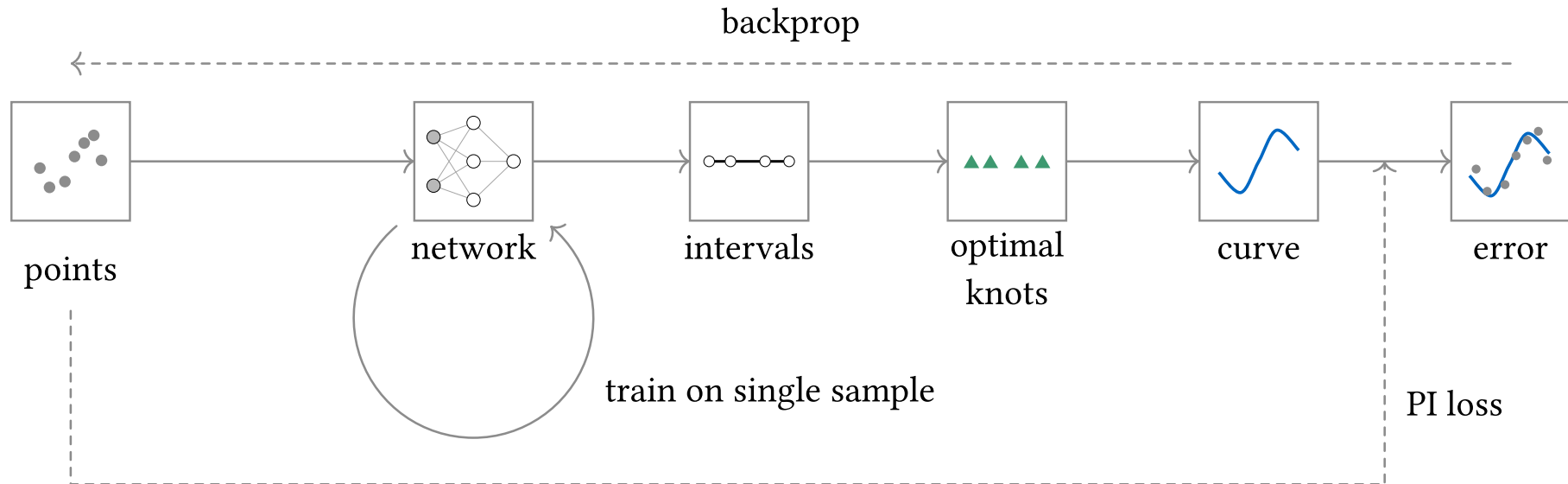
let's pretend
we don't
know this

DNN Solver



unsupervised training: only data points, no “ground truth” knots
physics inspired loss : $\|P - BC\|_F^2$

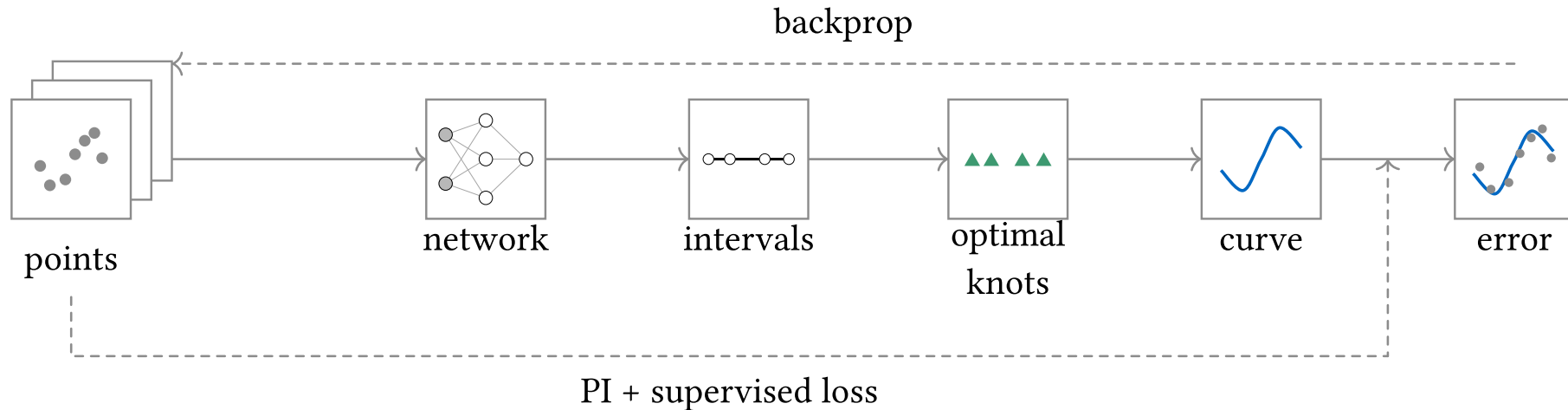
DNN Solver



unsupervised training: only data points, no “ground truth” knots

physics inspired loss : $\|P - BC\|_F^2$

Trained model



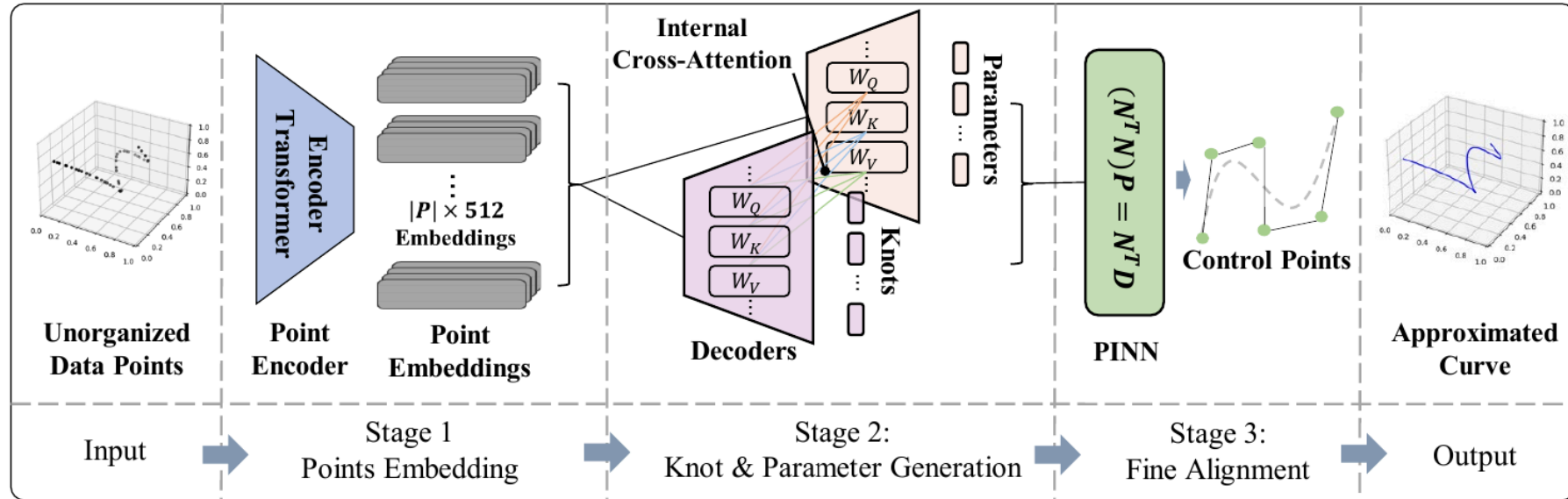
input: sampled B-spline curve + ground truth knots

Conclusions

- lukewarm results, better ML skills & more time could help
- ML-driven development (no need for domain knowledge)
 - e.g. sliding window ✗ curvature local feature
 - ✓ reduce number of input variables
 for more interpretable formulas
- AI + human collaboration also at meta level
(e.g. Copilot “auto-research”)

Future directions

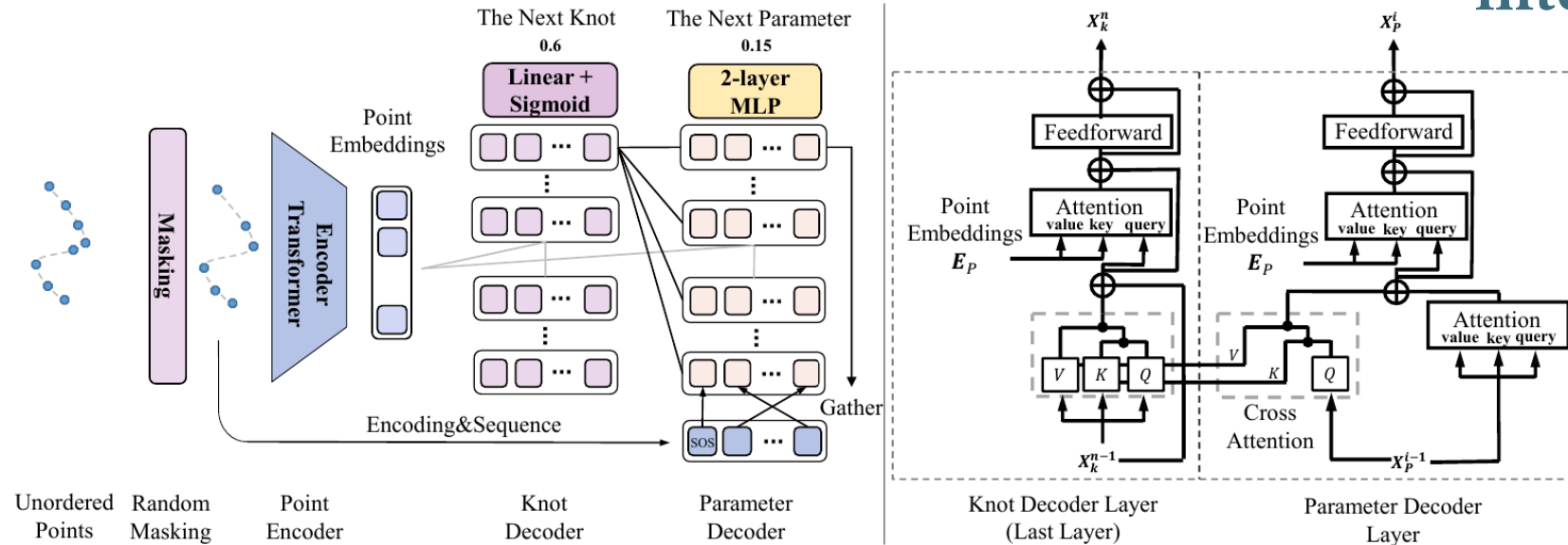
- try other tasks (e.g. knot insertion/removal, variable number of knots, knots-parametrization alignment, surface fitting)
- use actually good architectures, i.e. SplineGen



Future directions

- try other tasks (e.g. knot insertion/removal, variable number of knots, knots-parametrization alignment, surface fitting)
- use actually good architectures, i.e. SplineGen

how to make it interpretable???



SSPA

- ✓ docker + uv (docker-less setup on lab pc)
- ✓ bash, ssh, tmux
- ✓ git, pytest
- ✓ github actions & releases
- ✓ typst (compiler) ✗ make (so fast, just recompile everything)
- ✗ doxygen/sphinx (comments/docstrings are enough)
- ✗ parallel training (only 1 GPU)

SSPA

- ✓ docker + uv (docker-less setup on lab pc)
- ✓ bash, ssh, tmux
- ✓ git, pytest
- ✓ github actions & releases
- ✓ typst (compiler) ✗ make (so fast, just recompile everything)
- ✗ doxygen/sphinx (comments/docstrings are enough)
- ✗ parallel training (only 1 GPU)

Honorable mention: Copilot (in particular Claude Sonnet)
[RIP billing for request]